Web Fundamentals

WEF1VO

Teil 3: CSS-Grundlagen

Medientechnik und -design | WS 2025/26

1

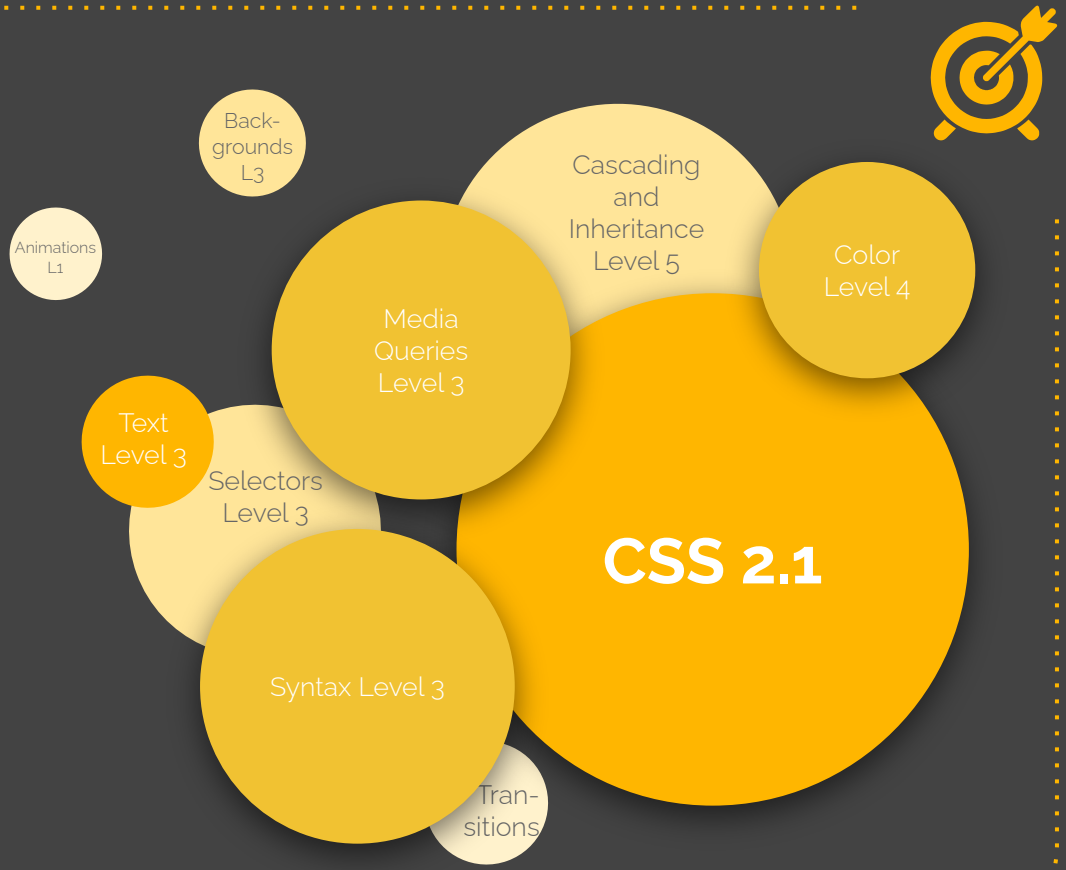
Cascading Style Sheets

Die Style-Sprache des Web

WHY CSS

CSS

Module & Versionen



Syntax

```
Selektor [, Selektor_2, ...] {
  Eigenschaft_1: Wert_1 [!important];
  Eigenschaft_2: Wert_2 [!important];
  ...
  Eigenschaft_n: Wert_n [!important];
}
```

Syntax Beispiele

```
h1 {
  color: green;
}
h2 {
  color: #FF0000;
  font-family: Arial !important;
}
h3, p {
  color: #0000FF; /* blue */
}
```

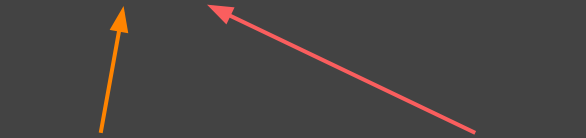
2

CSS Einbinden

Getting Style in Your Documents

Als Attribut – Inline Style

```
<h1 style="color: green;">
```



Als internes Stylesheet

```
<head>
  <style>
  h1 {
    color: green;
  }
  </style>
</head>
```

Vorteile?

Nachteile?

Als externes Stylesheet

```
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

Vorteile?

Nachteile?

Import weiterer Stylesheets

```
/* main.css */
@import url("base.css");
@import url("layout.css");
@import url("navigation.css");
```

3

CSS Selektoren

Was wie formatieren?

Element -/ Typ selektoren

```
body {
  background-color: #00FFFF;
}
h1 {
  color: #0000FF;
}
```

```
<body>
  <h1>Dies ist die Überschrift</h1>
  …
</body>
```

Universal selektor

```
* {
  color: #FF0000;
}
```

```
<body>
  <h1>Dies ist die Überschrift</h1>
  <p>Etwas <em>hervorgehobener</em> Text</p>
</body>
```

Klassen selektoren

```css
.big {
  font-size: xx-large;
  font-family: Arial, Helvetica, sans-serif;
}
p.crazy {
  color: #FF00FF;
  background-color: #00FF00;
}
```

```html
<p class="big">Absatz 1 schaut so aus.</p>
<p class="crazy">Absatz 2 etwas anders.</p>
```

ID-Selektoren

```
#primary {
  font-size: xx-large;
  color: red;
}
```

```
<p id="primary">Absatz mit einer ID.</p>
```

Attribut selektoren

```
a[href^="http://"] {
  color: #FF0000;
}
```

```
a[href^="http://" i] {
  color: #FF0000;
}
```

```
p[title]
input[type="text"]
p[class~="crazy"]
p[lang|="en"]
a[href^="http://"]
a[href$=".html"]
a[href*="TheHoff"]
```

```
<a href="http://speedtest.tele2.net/>HTTP-Link</a>
```

Nachfahr -Kombinatoren

```
div em {
  color: #FF0000;
  font-family: Arial, Helvetica, sans-serif;
}
```

```
<div>
  <p>Das ist der Text der <strong><em>hier</em></strong>
  formatiert wird.</p>
</div>
```

Kind-Kombinatoren

```
p > em {
  color: #FF0000;
  font-family: Arial, Helvetica, sans-serif;
}
```

```
<div>
  <p>Das wird <strong><em>hier nicht</em></strong>
  formatiert.</p>
</div>
<p><em>Hier</em> aber schon.</p>
```

Nächste Geschwister -K.

```
h1 + p {
  color: #FF0000;
  ft-family: Arial, Helvetica, sans-serif;
}
```

```
<h1>Hier die Überschrift</h1>
<p>Hier der erste, formatierte Absatz.</p>
<p>Hier der zweite, unformatierte.</p>
```

Nachfolg. Geschwister -K.

```
div ~ p {
  color: #FF0000;
  font-family: Arial, Helvetica, sans-serif;
}
```

```
<div>
  <p>Erster Absatz.</p>
</div>
<p>Zweiter Absatz.</p>
<p>Dritter Absatz.</p>
```

Pseudo elemente

```
p::first-line {
  color: fuchsia;
}
p::first-letter {
  font-size: xx-large;
}
p::selection {
  background-color: yellow;
}
```

```
p::before {
  content: "Text: ";
}
```

```
::after
::marker
::placeholder
```

Pseudo klassen

```
a:link {
  color: #FF0000;
  font-size: large;
}
li:nth-child(2n) {
  color: green;
}
section p:nth-of-type(2) {
  background-color: lightgray;
}
```

a·n+b Syntax (z.B. 5n+4)

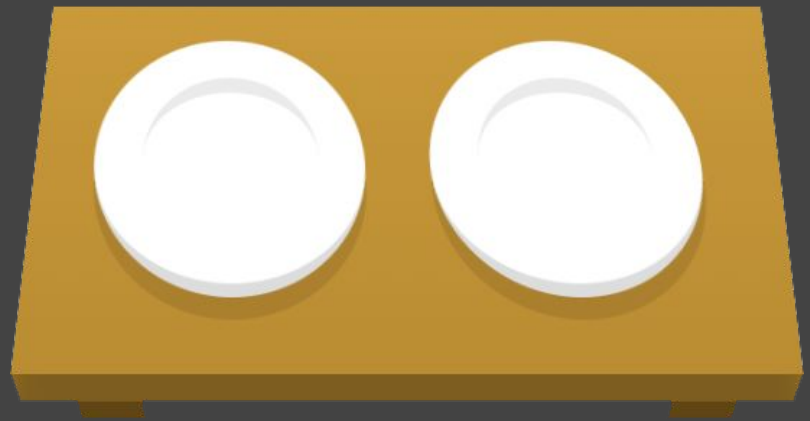
Funktionale Pseudo kl.

```
:is(h1, h2) {
  color: purple;
}

p:not(:first-child) {
  color: blue;
}

article:has(p) {
  background-color: lightgray;
}
```

CSS Diner



https://flukeout.github.io/

Vererbung in CSS

```
<p>Etwas Text, mit <em>wichtiger</em> Stelle</p>

p {
  color: #FF0000;
}
```

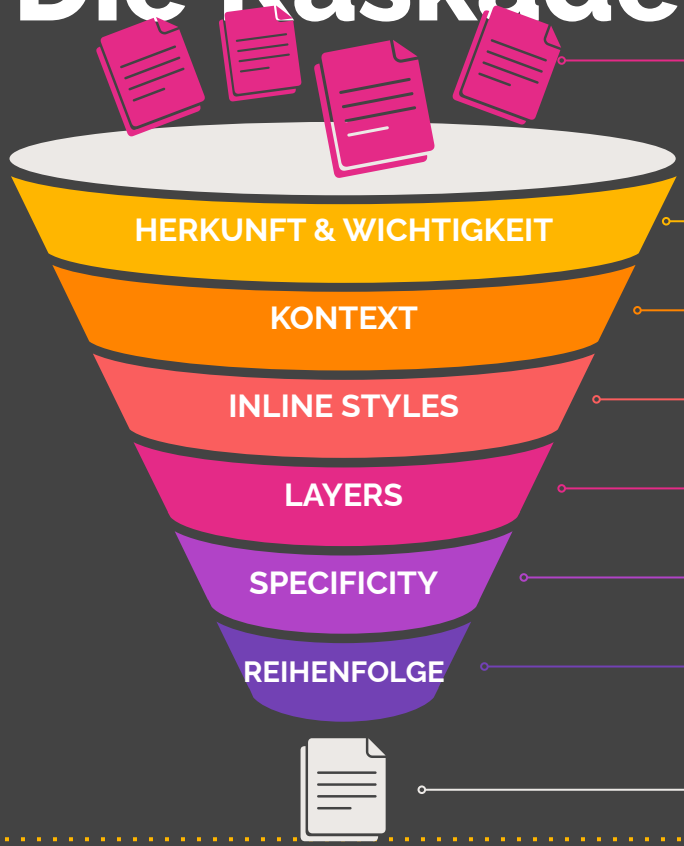
Ebenfalls rot?

4

Die Kaskade

Bestimmen, welche Regeln zum Zug kommen

Die Kaskade



Filterung
Finde alle gültigen Deklarationen für ein Element in allen Quellen und eingebundenen Stylesheets.

Aus welcher Art Stylesheet kommt die Regel und ist sie mit `!important` markiert? Genaue Reihenfolge!

Web-Komponenten können ihre eigenen Regeln mitbringen. Die Regeln der Seite gewinnen aber gegenüber diesen (bei !important ist es umgekehrt).

CSS-Angaben, die im HTML mit dem `style`-Attribut stehen, gewinnen vor Angaben in den Stylesheets.

CSS-Regel können mit `@layer` in Schichten aufgeteilt werden. Spätere Layer gewinnen vor früheren.

Spezifischere Deklarationen haben mehr Gewicht.

Spätere Deklarationen überschreiben frühere.

Konkrete Regel, wie das Element formatiert wird.

Ursprung & Wichtigkeit

1. Browser-Stylesheet
2. User-Stylesheet
3. Author-Stylesheet
4. CSS-Animationen
5. Author-Stylesheet !important
6. User-Stylesheet !important
7. Browser-Stylesheet !important
8. CSS-Transitions

Drei Stylesheets

Browser-Stylesheet

Auch Default-Stylesheet oder User Agent Stylesheet.

Vom Browser vorgegebene Standardwerte, wie Elemente aussehen.

User-Stylesheet

Von User*innen angelegtes Stylesheet, um selbst Anpassungen an Seiten vorzunehmen. Z.B. für Barrierefreiheit.

Schlecht unterstützt (nur in Firefox über userContent.css).

Author Stylesheet

Das Stylesheet, das mit der Webseite erstellt wird.

Womit wir hier arbeiten werden, meist nur als „Stylesheet“ bezeichnet.

Specificity

0,0,2

```
body h1 {
  color: green;
}
h1 {
  color: red;
}
```

0,0,1

?

IDs: 1,0,0

(Pseudo-) Klassen & Attr.: 0,1,0

(Pseudo-) Elemente: 0,0,1

Universalselektor, :where: 0,0,0

:is(), :has(), :not(): Spezifischter Inhalt

Drei Beispiele

```
li:first-child h2 .title {
  color: green;
}
```

0,2,2

```
#nav .selected > a:hover {
  font-weight: bold;
}
```

1,2,1

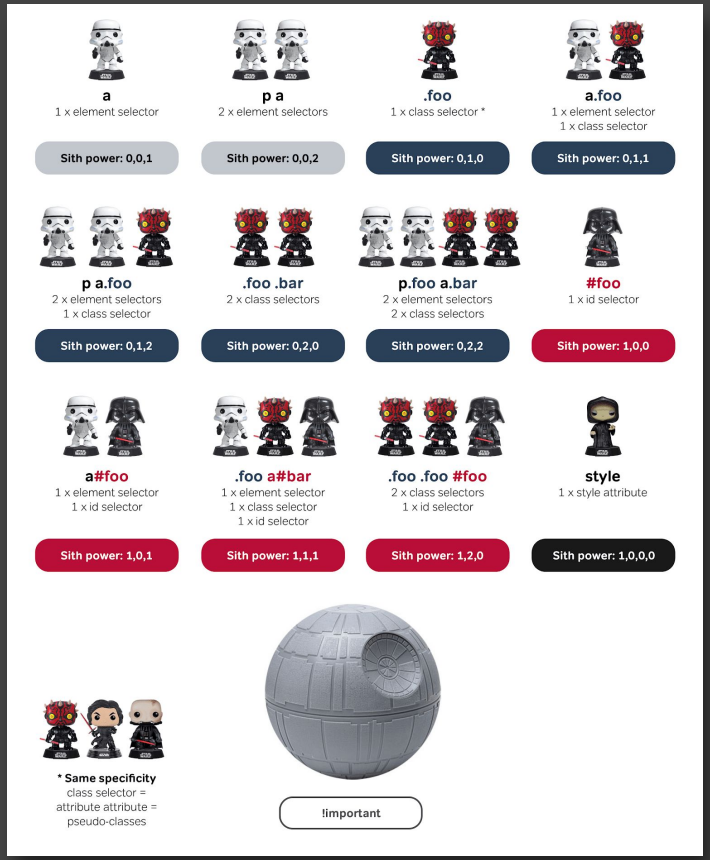
```
<p style="color: yellow">Text</p>
```

Eigene Ebene in der Kaskade!

1,0,0,0 (historisch)

Specificity Calculator

https://specificity.keegan.st/



5

Werte an CSS übergeben

Numerische Angaben, Text, Farben, Winkel & Zeit

Numerische Ang.

Absolut

- `mm:` Millimeter
- `cm:` Zentimeter (1 cm = 100 mm)
- `in:` Zoll (Inch, 1 in = 2,54 cm)
- `pt:` Punkt (Point, 1 pt = 1/72 in)
- `pc:` Pica (1 pc = 12 pt)
- `px:` Pixel (1 px = 0,75 pt)

Relativ

- `em:` Schriftgröße akt. Element
- `rem:` Schriftgröße Wurzelelement
- `ex:` Höhe des Buchstabens „x“
- `ch:` Breite des Zeichens „0“
- `lh:` Zeilenhöhe
- `rlh:` Zeilenhöhe Wurzelelement
- `%:` Prozentangabe

- `vw:` 1 % der Anzeigenbreite
- `vh:` 1 % der Anzeigenhöhe
- `vmin:` 1 % der kleineren Seite
- `vmax:` 1 % der größeren Seite
- `vb:` Viewport Block
- `vi:` Viewport Inline
- `svw:` Small Viewport Width
- `svh:` Small Viewport Height
- `lvw:` Large Viewport Width
- `lvh:` Large Viewport Height
- `dvw:` Dynamic Viewport Width
- `dvh:` Dynamic Viewport Height

Zeichenketten & Schlüsselwörter

```
p::before {
  content: "Text der davor eingefügt wird";
}

p {
  color: blue;
  font-weight: bold;
}
```

Farben

Benannte Farben

`black`, `maroon`, `green`, `olive`, `navy`, `purple`, `teal`, `silver`, `gray`, `red`, `orange`, `lime`, `yellow`, `blue`, `fuchsia`, `aqua`, `white`

Und 133 weitere ...

Hexadezimalschreibweise

RRGGBB	RRGGBB
`#FFFFFF`	`#0000FF`

Verkürzt:

`#FFF`	`#00F`

Farben

RGB

```
rgb(255 255 255);

rgb(0 0 255 / 0.5);
```

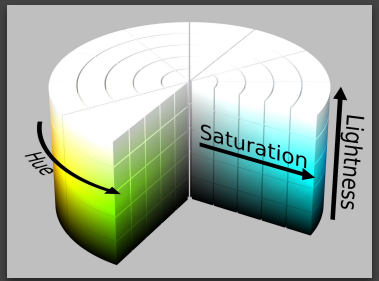
HSL

```
hsl(0 0% 100%);

hsl(240 100% 50% / 0.5);
```

Lab, Oklab, LCh, Oklch, LWB

```
lab(), oklab(), lch(),
oklch(), hwb()
```



Winkelmaße & Zeitangaben

Grad: `360deg`

Gon: `400grad`

Radiant: `6.28rad`

Vollwinkel: `1turn`

Sekunden: `1s`

Millisekunden: `1000ms`

6

CSS-Formate

Einige Beispiele (mehr in jeder CSS-Dokumentation)

Schriften formatierung

```
body {
  font-family: system-ui, "Segoe UI", Roboto,
               Arial, sans-serif;
  font-size: 16pt;
  font-weight: bold;
  font-style: italic;
  font-variant: small-caps;
}
```

Schriften definieren

```
@font-face {
 font-family: "Noto Sans";
 src: url("noto-sans-v42-latin-regular.woff2")
      format("woff2");
}
```

Formate: WOFF2, WOFF, TTF/OTF

Schriften definieren

```
@font-face {
  font-family: "Noto Sans";
  src: url("noto-sans-v42-latin-regular.woff2") format("woff2");
}

@font-face {
  font-family: "Noto Sans Italic";
  src: url("noto-sans-v42-latin-italic.woff2") format("woff2");
}

h1 {
  font-family: "Noto Sans", Arial, sans-serif;
}

p {
  font-family: "Noto Sans Italic", Arial, sans-serif;
}
```

Schriften Shorthand

```
body {
  font: bold italic small-caps 16pt
        system-ui, "Segoe UI", Roboto, Arial,
        sans-serif;
}
```

Text formatierung

```
p {
  text-decoration: line-through;
  text-transform: uppercase;
  color: maroon;
}
```

Abstände & Ausrichtung

```
p {
  text-indent: 1em;
  line-height: 1.5em;
  vertical-align: middle;
  text-align: justify;
  white-space: nowrap;
}
```

Hintergründe

```
body {
  background-color: aqua;
  background-image: url("background.webp");
  background-repeat: repeat-x;
  background-attachment: fixed;
  background-position: center 50px;
  background-clip: border-box;
  background-size: cover;
}
```

Hintergründe Shorthand

```
body {
  background: aqua url("background.webp")
              repeat-x fixed center 50px;
}
```

Listen formatierung

```
ul {
  list-style-type: square;
  list-style-position: inside;
  list-style-image: url("bullet.webp");
}
```

Listen Shorthand

```
ul {
  list-style: square url("bullet.webp")
              inside;
}
```

Nächstes Mal in WEF1VO

CSS Box-Modell & Responsive Web

Credits

- S. 4: CSS Logo: https://github.com/CSS-Next/logo.css
- S. 25: CSS Diner: https://flukeout.github.io/
- S. 33: CSS Specificity Wars: https://stuffandnonsense.co.uk/archives/css_specificity_wars.html
- S. 38: Color Circle (Hue-Sat): https://commons.wikimedia.org/wiki/File:Color_circle_(hue-sat).png
- S. 38: HSL color solid cylinder saturation gray: https://commons.wikimedia.org/wiki/File:HSL_color_solid_cylinder_saturation_gray.png